

RTL Design Sherpa

Pre-Bitstream Timing Prediction on FPGA Artix-7 Companion to the ASIC Methodology White Paper

June 12, 2026

Table of Contents

1 Readme Fpga....5

2 Pre-Bitstream Timing Prediction on FPGA (Artix-7)....6

2.1 Abstract....7

2.2 1. What changes between the ASIC paper and this one....8

2.3 2. The harness on FPGA....9

2.4 3. The Vivado flow....10

2.5 4. The spreadsheet....11

2.5.1 4.1 characterization....11

2.5.2 4.2 building blocks....12

2.6 5. The validation loop....13

2.6.1 5.1 Wrapper....13

2.6.2 5.2 Signature accumulator....13

2.6.3 5.3 On-board observation....14

2.6.4 5.4 Vivado STA contract....14

2.6.5 5.5 Quick command....14

2.6.6 5.6 The simpler sweep – when STA is enough....14

2.6.7 5.6a The same sweep on Quartus (Cyclone V)....15

2.6.8 5.7 What v0.1 still doesn't do....16

2.7 6. Recommended workflow....17

2.8 7. Limitations / intentional non-features....18

2.9 8. Navigation....19

List of Figures

No figures in this document.

List of Tables

No tables in this document.

1 Readme Fpga

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Pre-Bitstream Timing Prediction on FPGA (Artix-7)

The FPGA companion to the ASIC pre-synth methodology. Same harness, same nine FUBs, same spreadsheet shape – but the lookup numbers come from Vivado primitive datasheets (and eventually from real post-route reports off the Nexys A7), and the validation loop runs through an on-board clock sweep instead of an STA tool.

Version: 0.4 **Last Updated:** 2026-10-20 **Companion to:** `README.md` (ASIC / ASAP7 paper) **Status:** Vivado flow live; spreadsheet seeded with datasheet numbers; hardware validation loop planned, not built.

2.1 Abstract

This document carries the same purpose as the ASIC paper: **predict and close timing before the bitstream build ever starts**. On ASIC, that prediction is grounded in Yosys + ABC + OpenSTA per-gate-delay measurements. On FPGA the inputs are different but the workflow is the same:

- characterize the relevant Xilinx primitives (LUT6, CARRY4, DSP48, BRAM, MUXF7/F8, LUTRAM, SRL) at the speed grade you'll ship at,
- record a per-primitive ps-per-level row on the characterization sheet,
- the building-blocks sheet looks those numbers up via VLOOKUP and reports a slack column for every block you might instantiate,
- a designer at the spreadsheet sees fail / pass per block as they tweak the planned clock frequency, **before** committing to a 10-30 minute Vivado place-and-route round trip.

The deliverable is `work/artix7_characterization.xlsx`, generated by `work/build_fpga_characterization_xlsx.py`. The Vivado flow lives under `fpga/` – a Nexys A7-100T project that builds the same `char_top` harness, wrapped for a single board clock and eight observation LEDs.

This methodology is **explicitly proactive**. It is not a debugger to reach for after Vivado throws a negative WNS at you; it is a design-time budget table the designer consults *before writing the first `always_ff`*. The point is to flip the loop – characterize once per (device, speed grade), then build with a known-good budget – so place and route becomes a confirmation step rather than the first time anyone finds out the pipeline doesn't close.

2.2 1. What changes between the ASIC paper and this one

Topic	ASIC paper	FPGA paper
Process	TSMC-class ASAP7 PDK (open-source)	Xilinx Artix-7 (xc7a100tcsg324-1 on Nexys A7-100T)
Toolchain	Yosys + slang + ABC + OpenSTA	Vivado (synth + impl + bitgen)
Corners	TT / FF / SS (Liberty PVT corners)	Speed grade -1 / -2 / -3
Frequency range	2.0 GHz - 3.0 GHz (where 7nm starts to choke)	100 - 300 MHz (where Artix-7 typically choke)
Per-level primitives	NAND, XOR, MUX, CARRY, MULT, GRAY_CTR, QUEUE	LUT6, MUXF7, MUXF8, CARRY4, DSP48, LUTRAM, SRL
Mixed-bag proxy	MULT (multiplier tree)	DSP48 (DSP macro)
Reference single-point	INV, CLK_DIV	BRAM (RAMB36 sync read)
Tflop source	Tcq + Tsu derived from NAND chain probe	FDRE Tcq + Tsu (~400 ps -1, ~280 ps -3)
Wire delay model	per-metal-layer ps/mm (M2/M3 … M8/M9)	single FPGA-routing ps/hop (default 200 ps -1)
Ground truth	ABC `stime -c` per chain	Vivado post-route `report_timing_summary`
Validation loop	corner data already there (same Liberty)	on-board MMCM clock sweep + host-side golden model

The spreadsheet's **building blocks** sheet is identical RTL with FPGA- specific numbers swapped in. The **stream** sheet (per-FUB port walks) is intentionally omitted from this v0.1 – the per-port hand-walks port over 1:1 from the ASIC sheet, only the looked-up per-level numbers change.

2.3 2. The harness on FPGA

`char_top` is the same nine-FUB harness described in `README.md` Section 5. All anti-optimisation defences (LFSR-driven inputs, outputs routed to top-level ports, `dont_touch` attributes) work identically under Vivado.

For board-level use a thin wrapper trims the wide observation ports down to something the Nexys A7's 8 user LEDs can drive:

- `fpga/rtl/char_top_fpga.sv` instantiates `char_top` with all 9 FUBs enabled and XOR-reduces each FUB's output bus to a single bit.
- 8 LEDs drive one bit per FUB; the 9th FUB folds into LED[0] via an XOR with the NAND output so both chains reach a top-level port without losing an LED.
- A 100 MHz onboard oscillator (board pin **E3**) feeds `clk`; the CPU_RESETN button (**C12**) feeds `rst_n`; four switches (**J15** / **L16** / **M13** / **R15**) provide the LFSR seed nibble.

Every chain still reaches an output port, so Vivado optimisation can't delete anything – the FPGA equivalent of the ASIC scaffold.

2.4 3. The Vivado flow

The flow lives under `fpga/`, shaped like `stream_characterization/flows-stream-bridge` (retired):

```
fpga/
├── Makefile                       project | synth | bitstream |
program | clean
├── rtl/char_top_fpga.sv           thin wrapper around char_top
├── constraints/char_top_fpga.xdc Nexys A7-100T pin map + 100 MHz
clock
└── tcl/
    ├── create_project.tcl         project gen from
rtl/filelists/char_top.f
    ├── synth_only.tcl             synth + utilization_synth.txt
    ├── build_all.tcl              synth + impl + bitgen + reports
    └── (filelist expansion comes from the shared
make/tcl/filelist_utils.tcl)
```

Quick start (with `REPO_ROOT` exported via the repo's `env_python`):

```
cd projects/asic-trials/timing_characterization/fpga
make help            # see all targets
make synth           # ~2 min, posts utilization + post-synth timing
make bitstream       # ~10-30 min, full flow + all reports
make utilization     # cat the latest utilization report
make timing          # cat the latest post-route timing summary
make program         # flash if board is plugged in
```

Reports land under `fpga/reports/`:

File	Contents
`utilization_synth.txt`	post-synth LUT/FF/BRAM/DSP totals
`utilization_impl.txt`	post-route same
`timing_summary.txt`	per-path-group WNS, the calibration source
`timing_worst.txt`	top 50 worst paths
`power.txt`	Vivado's power estimate

The bitstream itself lands at `fpga/bitstream/timing_char.bit`.

2.5 4. The spreadsheet

`work/artix7_characterization.xlsx` is structured exactly like the ASIC companion. Two sheets:

2.5.1 4.1 `characterization`

Per-primitive ps-per-level rows for the seven primitives that scale with chain depth (LUT6, MUXF7, MUXF8, CARRY4, DSP48, LUTRAM, SRL), plus a single-point reference for BRAM access time. Speed grades -1 / -2 / -3 across the five-frequency sweep (100 / 150 / 200 / 250 / 300 MHz).

Sample TT-like numbers (Artix-7 -1 ballparks):

primitive	per-level ps (-1)	notes
LUT6	200	single-LUT propagation
MUXF7	250	F7 cascade
MUXF8	280	F8 cascade
CARRY4	80	per-bit CIN -> COUT (dedicated chain)
DSP48	**2500**	**PROXY for mixed-bag logic**
LUTRAM	600	distributed RAM read
SRL	400	SRL16 / SRL32 shift
BRAM (ref)	2400	RAMB36 sync read, 1-cycle

Tflop (FDRE Tcq + Tsu): 400 ps at -1, 340 ps at -2, 280 ps at -3.

The **Design clocks table** at the bottom is the single planning point. One row per clock the design uses:

clock name	freq (MHz)	target sg	period	input 60%	output 40%	wire/hop
clk_main	**100**	**sg-1**	derived	derived	derived	200
(5 empty slots)						

Yellow cells are editable; everything else derives. The “wire/hop” column defaults higher than the ASIC sheet (200 ps -1) because FPGA routing dominates timing more than ASIC standard-cell routing.

2.5.2 4.2 `building blocks`

One row per reusable RTL block, with editable params and corner-aware target columns:

- `gaxi_skid_buffer`
- `gaxi_fifo_sync (REG=0)`
- `gaxi_fifo_sync (REG=1, BRAM-backed)`
- `arbiter_round_robin`
- `axi4_master_rd`
- `axi4_master_wr`
- `BRAM (raw RAMB36)`

Each row’s slack cell pulls from the Design clocks table via VLOOKUP – change `clk_main`’s freq or speed grade and slack flips per block. Negative slack renders **bold red**, three-decimal precision (`0.000`).

2.6 5. The validation loop

The ASIC paper has STA as the ground truth – Yosys + ABC + OpenSTA and the delay numbers come out of `stime -c`. On FPGA the equivalent is Vivado's post-route `report_timing_summary`, which `make bitstream` already emits. The on-board sanity check that "does the spreadsheet match what the chip actually does at this clock?" is the **MMCM sweep**:

> **Honest assessment up front.** On Artix-7 at the frequencies in this sweep (100–300 MHz), the FUBs are small enough that Vivado's STA is generally trustworthy, and the simpler `bitstream-sweep` target (§5.6) gives you everything you need to calibrate the spreadsheet. The MMCM sweep adds little beyond that – it only earns its keep on the rare outlier path where STA mis-modelled the delay and silicon disagrees. Keep it as the "is it real?" gut check; do not treat it as the headline calibration loop. Most users should reach for `make bitstream-sweep`, not `make bitstream-mmcm`.

2.6.1 5.1 Wrapper

`fpga/rtl/char_top_fpga_mmcm.sv` wraps an `MMCME2_BASE` primitive with a VCO running at 1200 MHz, exposing four test clocks to four parallel `char_top` instances:

domain	divide	test freq
`clk_test_0`	/8	150 MHz
`clk_test_1`	/6	200 MHz
`clk_test_2`	/5	240 MHz
`clk_test_3`	/4	300 MHz

Each instance gets the same LFSR seed nibble from `SW[1:0]`, so all four domains are doing the same work – just at different clocks. Per-domain reset is held until the MMCM locks, then released through a 2-flop synchroniser into each test clock.

2.6.2 5.2 Signature accumulator

`fpga/rtl/sig_accum.sv` folds every char_top output bus (207 bits worth – nand, inverter, xor, carry, mult, mux, queue, clk_div, gray) into a 32-bit XOR signature in the source domain. A free-running snapshot register updates every 2^16 source-clock cycles and the value crosses into the 100 MHz observation domain via a 2-flop CDC synchroniser (Xilinx `ASYNC_REG = "TRUE"` attributes preserve the metastability margin).

2.6.3 5.3 On-board observation

Eight user LEDs show:

LED	meaning
LED[0]	clk_test_0 (150 MHz) signature changed since last snapshot
LED[1]	clk_test_1 (200 MHz) signature changed
LED[2]	clk_test_2 (240 MHz) signature changed
LED[3]	clk_test_3 (300 MHz) signature changed
LED[7:4]	lower 4 bits of the signature for the selected clock

SW[3:2] selects which domain drives LED[7:4] for closer inspection.

A failing clock domain's signature freezes (LED stays solid one way or the other instead of flickering at a few Hz), so a quick glance at LED[3:0] tells you which test clocks the chip is keeping up with.

2.6.4 5.4 Vivado STA contract

fpga/constraints/char_top_fpga_mmcm.xdc declares each MMCM output as a generated clock and groups them as asynchronous to the 100 MHz observation clock, so make bitstream-mmcm produces a timing_summary.txt with one WNS row per clk_test_N group. Those WNS numbers are the calibration source: feed them back into work/build_fpga_characterization_xlsx.py's PRIM_PS dict and regenerate the workbook.

2.6.5 5.5 Quick command

```
cd projects/asic-trials/timing_characterization/fpga
make bitstream-mmcm   # builds char_top_fpga_mmcm with 4 test clocks
make program          # flash the board
                      # watch LED[3:0] for clock-domain liveness
```

2.6.6 5.6 The simpler sweep – when STA is enough

The MMCM build is only worth the extra silicon when you want to **observe on-chip behaviour** at multiple clocks from one bitstream. If you only need Vivado's STA opinion to feed the spreadsheet, the simpler flow is the bitstream-sweep target – it loops the regular char_top_fpga wrapper across a list of target periods, archives each run's reports under reports/sweep_<F>MHz/, and rolls per-clock-group WNS into a single reports/sweep_wns.csv:

```
make bitstream-sweep FREQS_MHZ="100 150 200 250 300"
# ... runs N back-to-back full builds ...
cat reports/sweep_wns.csv
```

CSV columns: `freq_mhz, period_ns, group, wns_ns, slack_status, utilization_luts, utilization_ffs, utilization_brams, utilization_dsps, data_delay_ns, logic_levels` (the last two are filled by the Quartus sweep below and empty here). That's the direct input to recalibrating `work/build_fpga_characterization_xlsx.py`'s `PRIM_PS` dict.

Under the hood it's just a `TARGET_PERIOD_NS` env var that `fpga/tcl/clock_constraint.tcl` reads at project-creation time and bakes into a fresh XDC fragment. No MMCM, no sig_accum, no on-chip behaviour – pure post-route Vivado STA.

2.6.7 5.6a The same sweep on Quartus (Cyclone V)

`make quartus-sweep` is the Quartus twin (timing_characterization TASK-001, 2026-09-29). It compiles `char_top` itself – not a board wrapper – once per frequency on the device named in a config file, with the SAME multi-flow SDC the ASIC and Vivado flows use (`rtl/syn/char_top.sdc`, `FLOW "quartus"`), so the three flows measure one constraint:

```
cd fpga
make quartus-sweep
QUARTUS_SH=~/intelFPGA_lite/24.1std/quartus/bin/quartus_sh
#   QUARTUS_CFG=quartus/sweep_config.example.tcl   (device, I/O split, parameters)
#   FREQS_MHZ="100 150 200 250 300"                (same default list as bitstream-sweep)
make quartus-sweep-dry   # print the plan under plain tclsh, compile nothing
cat reports/quartus/sweep_wns.csv
```

Each point is a fresh project under `build/quartus/<F>MHz/` (map, fit, STA; no assembler, ~90 s on a Cyclone V GX for the full char_top). Two things the Vivado sweep does not give you land in `reports/quartus/sweep_<F>MHz/`:

- **`fub_slack.csv`** from `fpga/quartus/sta_reports.tcl`: for every FUB the worst register-to-register setup path launched from its flops – slack, data-path delay and **logic levels** – plus a `<fub>.io` row for the worst path to any endpoint (the 20 % output-delay budget lands on those) and a `design` row. This is the per-FUB gate-delay signal the methodology is after, read straight from the Timing Analyzer's `get_timing_paths` rather than eyeballed from a report.
- **All four timing models** (slow/fast x 85C/0C) as `clk:<model>` rows.

`parse_timing_sweep.py --tool quartus` rolls both into the same CSV shape as the Vivado sweep, with two extra columns: `data_delay_ns, logic_levels` (empty on Vivado rows). `utilization_luts` carries ALMs on Quartus rows – an ALM is not a LUT; compare across tools by ratio, not by count.

Config is a plain Tcl file of `set` statements (`fpga/quartus/sweep_config.example.tcl` documents every key). The default device is the Cyclone V GX Starter Kit part, 5CGXFC5C6F27C7: the DE1-SoC's 5CSEMA5F31C6 does not have 242 user pins free for the fully-enabled char_top; set `VIRTUAL_PINS 1` to use it. Quartus Prime Lite 24.1std installs only Cyclone V and MAX 10.

Two things the first Quartus run taught, both now in the SDC: the Timing Analyzer does not implement `set_load`, and its `set_clock_uncertainty` names the clock with `-to` and takes the value last – `read_sdc` aborts on the first error and the fitter then reports "Can't fit design", which is the SDC failure wearing a resource costume.

2.6.8 5.7 What v0.1 still doesn't do

- **Host-side golden model.** Both sweeps (MMCM and target-freq) report structural / STA signals. Neither asks "did the chip produce THE RIGHT data?" A UART or USB-blaster handshake out to the host's LFSR reference model would close that gap.
- **Spreadsheet auto-update.** `parse_timing_sweep.py` produces `sweep_wns.csv` but doesn't feed the values back into the workbook builder's `PRIM_PS` dict; that's still a copy-paste workflow.

2.7 6. Recommended workflow

1. **Plan**. Open `work/artix7_characterization.xlsx`. On `characterization`, set `clk_main` freq + speed grade to your target. Flip to `building blocks` – any red rows are the blocks you'll need to pipeline harder or use sparingly.
2. **Iterate the plan**. Lower the freq until everything's green (the design's actual ceiling on this device); switch the speed grade to model a faster-graded part; bump `wire/hop` to model longer inter-CLB routes.
3. **Build**. `cd fpga; make synth` for a quick utilisation check; `make bitstream` for the full place-and-route + reports.
4. **Calibrate**. Read `reports/timing_summary.txt` after `make bitstream`. Compare the per-FUB WNS against what the spreadsheet predicted. Update `work/build_fpga_characterization_xlsx.py`'s `PRIM_PS` dict with the calibrated numbers and regenerate the workbook.

The methodology pays for itself the first time it tells you "your plan doesn't fit at 300 MHz on -1 silicon" before you spend 30 minutes on a Vivado run to learn the same thing.

2.8 7. Limitations / intentional non-features

- **Seed data are datasheet ballparks**, not calibrated measurements. Treat the v0.1 workbook as a methodology demo; the per-LUT / per-CARRY4 numbers are reasonable to start a discussion but should be replaced with calibrated values from real Vivado runs.
- **Wire delay is one number.** ASIC has four metal layers in the workbook; FPGA's interconnect is more uniform and the inter-CLB routing dominates. One ps/hop with a per-clock override is enough resolution for a SWAG.
- **No MMCM sweep, no host golden model.** The on-board validation loop in section 5 is sketched but not wired up.
- **No power numbers.** Vivado emits `report_power`; the workbook doesn't yet model it.

2.9 8. Navigation

- ASIC companion paper: `README.md`
- Vivado flow Makefile: `fpga/Makefile`
- Board pin map: `fpga/constraints/char_top_fpga.xdc`
- FPGA wrapper RTL: `fpga/rtl/char_top_fpga.sv`
- FPGA spreadsheet: `work/artix7_characterization.xlsx`
- Spreadsheet generator: `work/build_fpga_characterization_xlsx.py`
- Components area: `../../components/README.md`
- Repository guide: `/CLAUDE.md`

Version: 0.4 **Last Updated:** 2026-10-20 **Maintained By:** RTL Design Sherpa Project